МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа
Дисциплина **Технологии распространения, развёртывания и сопровождения ПО**
наименование дисциплины

УЧЕБНО-ПРАКТИЧЕСКАЯ РАБОТА
№2
номер задания (при наличии)

Процессы непрерывной интеграции и непрерывной поставки (CI/CD)
Вариант 17
при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О727Б

Тарасевич В.В.
подпись фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Мажайцев Е.А.
подпись фамилия и инициалы

Оценка / балльная оценка

дата проверки

г. Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
ПРИЛОЖЕНИЕ А Листинг кода 9
А.1 Листинг кода CI/CD скриптов 9
А.2 Листинг кода тестов 9
А.3 Листинг github-ci.yml 10
А.4 Ссылка на репозиторий GitHub 11

1 Постановка задачи

Требуется реализовать сборочную линию процесса непрерывной интеграции (CI – Continuous Integration). Для достижения этой цели требуется решить задачи, представленные ниже.

Реорганизация репозитория – требуется реорганизовать структуру репозитория, чтобы она удовлетворяла требованиям для создания deb-пакета: в корне репозитория располагаются директории DEBIAN, cicd, src, usr/bin. В директории DEBIAN располагается файл control с мета-данными пакета. В директории cicd располагаются скрипты для сборки, тестирования и упаковки ПО. В директории src располагаются исходные файлы ПО: Makefile, main.cpp. В директории usr/bin располагается собранное ПО. Помимо этого, необходимо подготовить репозиторий для интеграции с механизмом GitHub Actions: создать по пути .github/workflows/ файл github-ci.yml, который будет управлять всей сборочной линией.

Написание скриптов для сборки, тестирования и упаковки ПО – требуется разработать shell-скрипты, производящие сборку программы через Makefile, проводящие тесты собранного ПО и формирующие из собранного ПО deb-пакет. Скрипты должны запускаться из файла github-ci.yml.

Настройка сборочной линии – необходимо сконфигурировать файл github-ci.yml, управляющий всей сборочной линией. Данный workflow должен запускаться при команде git push в главную ветку main. В workflow должны быть описаны процессы сборки ПО на раннерах GitHub, его тестирования и упаковки в deb-пакет, который, в последствие, загружается в выходной артефакт workflow.

2 Выполнение работы

Прежде всего, репозиторий был преобразован в соответствии с требованиями к deb-пакетам, что представлено на рисунке 1.

```
deralord@UbuntuDevops:~/Desktop/devops$ tree
.
├── cicd
│   ├── build.sh
│   ├── deploy.sh
│   ├── Makefile
│   ├── tests.cpp
│   └── test.sh
├── DEBIAN
│   └── control
├── README.md
├── src
│   ├── main.cpp
│   └── Makefile
└── usr
    └── bin
        └── sentrim
```

Рисунок 1 – Директория репозитория

После создания всех необходимых директорий и файлов, были разработаны тесты и необходимые для CI/CD скрипты, а директория cicd приняла вид, представленный на рисунке 2.

```
deralord@UbuntuDevops:~/Desktop/devops$ tree
.
├── cicd
│   ├── build.sh
│   ├── deploy.sh
│   ├── Makefile
│   ├── tests.cpp
│   └── test.sh
```

Рисунок 2 – Созданные скрипты и тесты

Shell-скрипты описывают процессы сборки, тестирования и упаковки ПО, тесты же описываются в файле tests.cpp. Листинг всех файлов представлен в приложении А.

После написания необходимых скриптов и тестов, был сконфигурирован файл управления сборочной линией github-ci.yml, листинг которого представлен в приложении А. На рисунке 3 представлена выдержка из этого файла, описывающая процесс сборки ПО.

```
  GNU nano 7.2                    github-ci.yml
name: CI-Pipeline

on:
  push:
    branches: main
  pull_request:
    branches: main

jobs:
  Build:
    runs-on: ubuntu-latest

    steps:
      - name: Clone repo
        uses: actions/checkout@v4

      - name: Install dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y build-essential g++ make

      - name: Build program
        run: ./build.sh
        working-directory: ./cicd

      - name: Upload binary to artifact
        uses: actions/upload-artifact@v4
        with:
```

Рисунок 3 – Сниппет github-ci.yml

Как видно из рисунка 3, файл описывает конфигурацию и инструкции, которые в последствие будут выполнены на удалённом раннере. Так, на рисунке 3 представлено начало файла и этап сборки. Видно, что линия будет активироваться только при git push или при открытии пул-реквеста в ветку main. Сборка запускается на последней версии ОС Ubuntu. Раннер клонирует в свою файловую систему репозиторий, устанавливает необходимые для сборки ПО пакеты, после чего запускает скрипт cicd/build.sh, в котором описывается запуск Makefile и сборка приложения в директорию usr/bin. После этого, раннер формирует артефакт, в который помещается собранный до этого исполняемый файл. Это требуется для следующего этапа

тестирования. Каждый этап выполняется отдельно, на «чистой» системе, потому требуется использовать такой механизм как артефакты, чтобы передавать данные между этапами.

По аналогии выполняются следующие два этапа: тестирование и упаковка. GitHub позволяет отслеживать ход работы сборочной линии на странице с удалённым репозиторием, что представлено на рисунке 4.

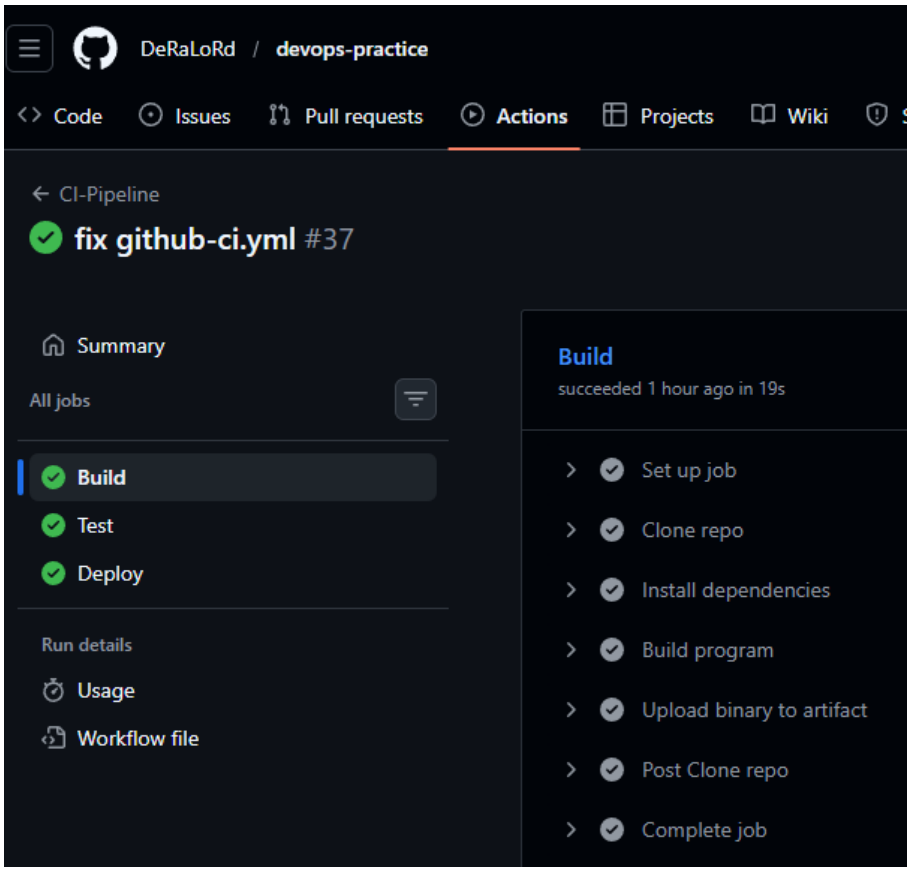

Рисунок 4 – Страница сборочной линии в GitHub

Как можно видеть, в GitHub выполняются те же шаги, что были описаны до этого в github-ci.yml.

Сборочная линия настроена так, что следующий этап не будет отправлен на выполнение, пока не завершится предыдущий, что логично. Таким образом, если программа успешно соберётся, но не пройдёт тестирование, она не будет отправлена на упаковку, что представлено на рисунке 5.

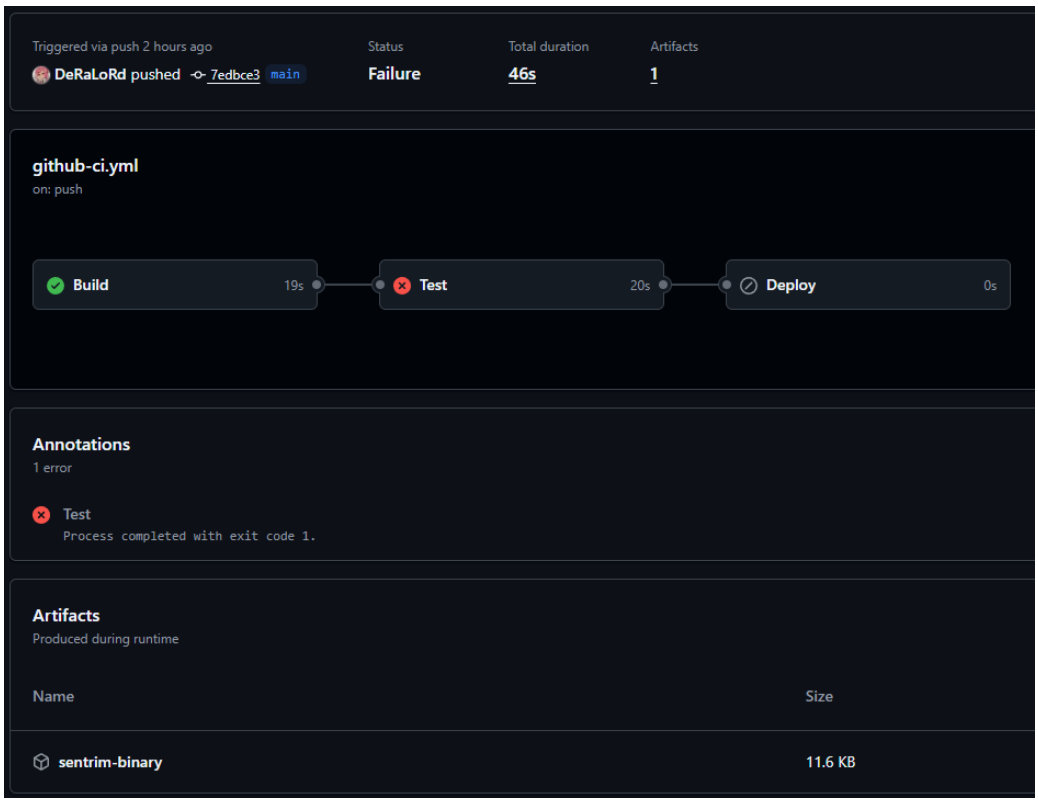

Рисунок 5 – Ошибка при тестировании

На рисунке 5 видно, что сборка прошла успешно, но тесты не были пройдены. В виду этого этап упаковки не был выполнен, и сборочная линия была остановлена. Для сравнения, на рисунке 6 представлено успешное прохождение всех этапов.

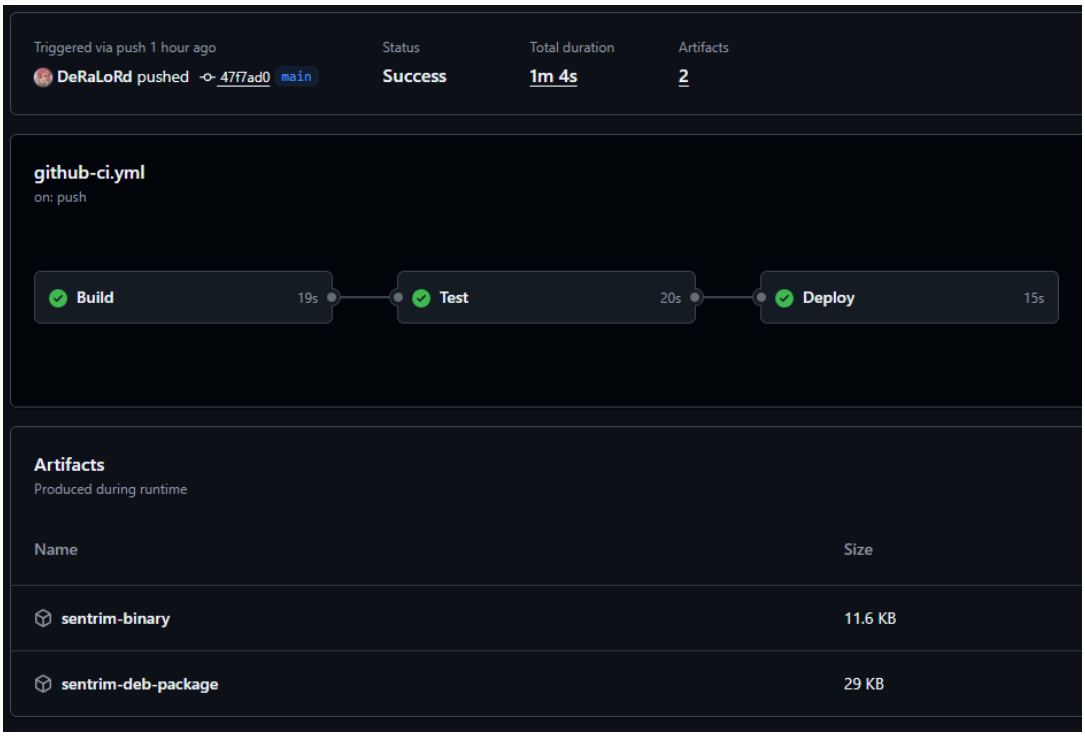

Рисунок 6 – Успешное выполнение всех этапов

Так как все этапы были выполнены успешно, ПО было упаковано в deb-пакет и отправлено в выходной артефакт: sentrim-deb-package. Таким образом, удалось настроить сборочную линию через GitHub Actions, которая при пуше в удалённый репозиторий автоматически производит сборку ПО, проводит тестирование и, при успешном прохождении тестов, упаковывает ПО для дальнейшего скачивания.

ПРИЛОЖЕНИЕ А

Листинг кода

А.1 Листинг кода CI/CD скриптов

Ниже представлен листинг файла build.sh.

```
#!/bin/bash
# Script is in repo/cicd directory
# Create repo/usr/bin folder
mkdir -p ../usr/bin
# Go to repo/src and run Makefile there
make -C ../src
```

Ниже представлен листинг файла test.sh.

```
#!/bin/bash
# Script is in repo/cicd directory
# Compile tests from tests.cpp
make
# Start the tests
./sentrim-tests
```

Ниже представлен листинг файла deploy.sh.

```
#!/bin/bash
# Script is in repo/cicd directory
# Create deb-package with passed name
dpkg-deb --build .. $1
```

А.2 Листинг кода тестов

Ниже представлен листинг файла tests.cpp.

```
#include <iostream>
#include <string>
using namespace std;
int run_test(string name, string input, string expect);
int main(int argc, char *argv[])
{
	int result;
	result += run_test(
		"Test 1",
		"to be or not to be. thats the question.",
		"not. question. "
	);
	result += run_test(
		"Test 2",
		". .. f .see omg to lol.",
		"f. see omg lol. "
	);
	result += run_test(
		"Test 3",
		"",
		""
	);
	return result;
}
int run_test(string name, string input, string expect)
{
	string command = "echo \"" + input + "\" | ../usr/bin/sentrim";
	//cout << "EXECUTING COMMAND: " << command << '\n';
	FILE* pipe = popen(command.c_str(), "r");
```

```
        if (!pipe) {
              throw runtime_error("popen failed");
        }
        char buffer[128];
        string result;
        while (fgets(buffer, sizeof(buffer), pipe) != NULL) {
              result += buffer;
        }
        pclose(pipe);
        // remove '\n' from result
        result = result.substr(0, result.find('\n'));
        //cout << "got: " << result << '\n';
        //cout << "expected: " << expect << '\n';
        if (!result.compare(expect)) {
              cout << name << " passed\n";
              return 0;
        }
        cout << name << " failed\n";
        return 1;
}
```

A.3 Листинг github-ci.yml

Ниже представлен листинг файла github-ci.yml.

```
name: CI-Pipeline

on:
  push:
    branches: main
  pull_request:
    branches: main

jobs:
  Build:
    runs-on: ubuntu-latest

    steps:
      - name: Clone repo
        uses: actions/checkout@v4

      - name: Install dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y build-essential g++ make

      - name: Build program
        run: ./build.sh
        working-directory: ./cicd

      - name: Upload binary to artifact
        uses: actions/upload-artifact@v4
        with:
          name: sentrim-binary
          path: usr/bin/sentrim
          retention-days: 1

  Test:
    needs: Build
    runs-on: ubuntu-latest

    steps:
      - name: Clone repo
        uses: actions/checkout@v4
```

```
      - name: Install dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y build-essential g++ make tree

      - name: Create usr/bin/ directory for binary
        run: mkdir -p usr/bin
        working-directory: .

      - name: Download binary from artifact
        uses: actions/download-artifact@v4
        with:
          name: sentrim-binary
          path: usr/bin

      - name: Give permissions to binary
        run: chmod +x sentrim
        working-directory: ./usr/bin

      - name: Run tests
        run: ./test.sh
        working-directory: ./cicd

  Deploy:
    needs: Test
    runs-on: ubuntu-latest

    steps:
      - name: Clone repo
        uses: actions/checkout@v4

      - name: Install dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y dpkg-dev

      - name: Create usr/bin directory
        run: mkdir -p usr/bin
        working-directory: .

      - name: Download binary from artifact
        uses: actions/download-artifact@v4
        with:
          name: sentrim-binary
          path: usr/bin

      - name: Build package
        run: ./deploy.sh "../sentrim_1.0_linux_amd64.deb"
        working-directory: ./cicd

      - name: Upload package
        uses: actions/upload-artifact@v4
        with:
          name: sentrim-deb-package
          path: sentrim_1.0_linux_amd64.deb
```

A.4 Ссылка на репозиторий GitHub

https://github.com/DeRaLoRd/devops-practice.